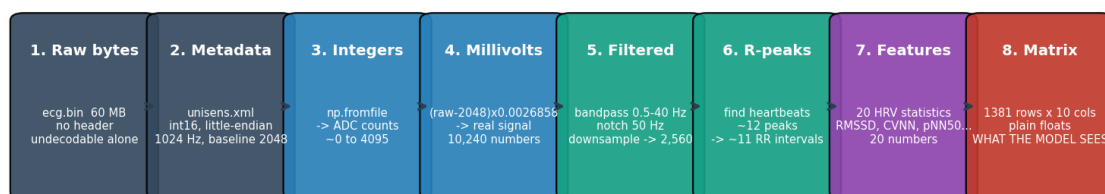


Detecting Atrial Fibrillation from Wearable ECG

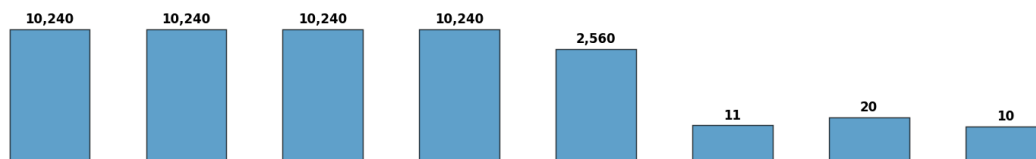
A Complete Step-by-Step Study Guide

HRV Feature Extraction and Context-Aware False Positive Analysis

How the ECG dataset becomes numbers a model can read



How much data survives each step (per 10-second segment)



10,240 raw samples -> 10 selected features. A ~1000:1 reduction.

Every step deliberately throws information away. This is why a bug at step 6 cannot be repaired at step 8 - the information is already gone.

The whole process in one picture: raw sensor bytes become ten numbers a model can read.

Dataset	CACHET-CADB (Context-Aware Cardiac Holter Database)
Segments analysed	1,602 labelled 10-second ECG segments
Subjects	23 (20 cardiac patients + 3 healthy controls)
Best result	AUC 0.932 (SVM), 0.923 (XGBoost)
Main finding	78-84% false-alarm rate on motion-corrupted signal
Status	Phases 1-8 complete; quality gate evaluated

*Written for a reader who is new to biomedical signal processing.
Every technical term is explained the first time it appears.*

What Is In This Guide

Part	Topic	What you will learn
0	System architecture	The whole pipeline on one page, layer by layer
1	The medical problem	What atrial fibrillation is and why detecting it matters
2	The dataset	What CACHET-CADB contains and how it is organised
3	Bytes to numbers	How raw sensor files become a table a model can read
4	Cleaning the signal	Filtering, and why we throw information away on purpose
5	Finding heartbeats	R-peak detection and RR intervals
6	Measuring irregularity	The 20 HRV features and what each one means
7	Five bugs we found	Errors that would have invalidated everything
8	Training the models	SVM, XGBoost, and honest cross-validation
9	The results	What the models achieved
10	Why it fails	The false-positive investigation - the real contribution
11	Attempting a fix	The quality gate, and why it cannot work
12	Where we stand	Current status and what comes next
12b	Related work	How this compares to a published 2025 paper
13	Glossary	Every term, defined plainly

The System at a Glance

Before working through the parts one at a time, here is the whole system on a single page. Read it top to bottom: raw sensor files enter at the top, and research findings come out at the bottom. Each horizontal band is one layer, and every box is a real file in the project.

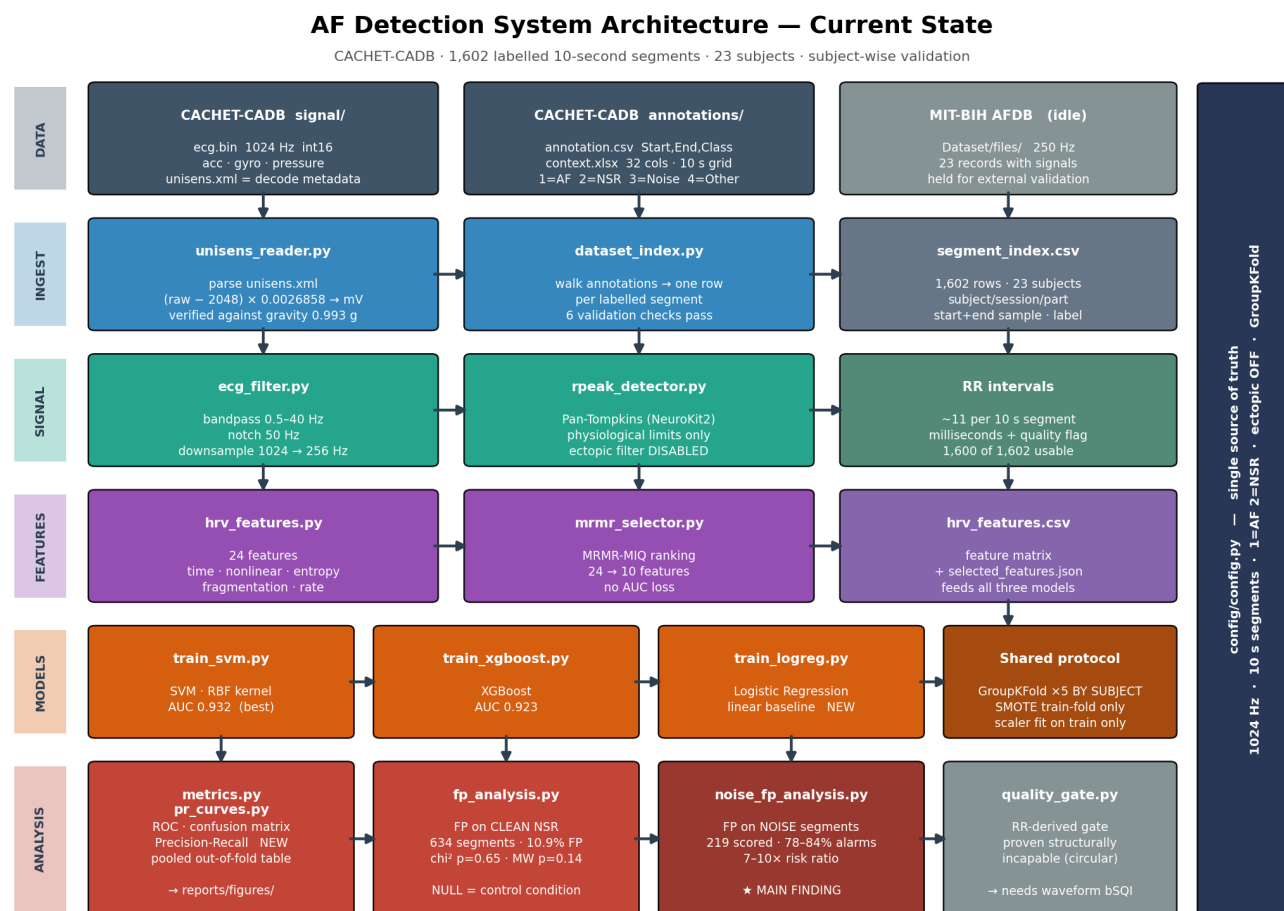


Figure 0.1 — System architecture. Six layers: data, ingest, signal processing, features, models, analysis. The dark column on the right is the configuration file, which every layer reads from.

Three design decisions this diagram is making visible:

- **Configuration is a spine, not a step.** config.py runs down the right-hand side because every layer reads from it. The five facts that were hardest to establish — the 1024 Hz sample rate, the 10-second segment length, the 1 = AF mapping, the disabled ectopic filter, and subject-wise splitting — live in exactly one place, so they cannot drift apart.
- **The three models share one protocol.** The box beside them is not a fourth model; it is the identical GroupKFold split, SMOTE placement and scaling applied to all three. Holding the protocol constant is what makes the comparison meaningful rather than accidental.
- **Two of the four analysis boxes are negative results, on purpose.** The clean-NSR analysis returns a null, and that null is the control condition that makes the noise result interpretable. The quality gate is shown greyed out because it was proven structurally incapable, not because it is unfinished. Keeping a disproved branch visible is more honest than deleting it.

The MIT-BIH AFDB box at the top right is greyed because it is downloaded but unused. It is reserved for external validation, which is outstanding work described in Part 12.

Part 1 — The Medical Problem

Your heart has four chambers. The two upper ones are the **atria**; the two lower ones are the **ventricles**. In a healthy heart, a natural pacemaker called the sinus node sends one clean electrical signal at a steady rhythm. The atria squeeze, then the ventricles squeeze. This is called **normal sinus rhythm**, abbreviated NSR.

In **atrial fibrillation** (AF), that orderly signal breaks down. The atria quiver chaotically, firing hundreds of disorganised impulses per minute. The ventricles respond to whichever impulses get through, so they beat at unpredictable intervals. A doctor describes this as **irregularly irregular** — not just fast or slow, but with no pattern at all.

Why this matters: when the atria quiver instead of squeezing, blood pools inside them and can clot. If a clot travels to the brain it causes a stroke. AF raises stroke risk roughly fivefold and affects over 37 million people worldwide. Many do not know they have it, because it can come and go without symptoms.

This creates the engineering problem. Catching intermittent AF means monitoring someone for days while they go about normal life — walking, working, sleeping. That means a wearable sensor, not a hospital machine. And a wearable records whatever the body does to it: movement, sweat, electrodes shifting against skin.

The one thing we can measure

Here is the insight the whole project rests on. We do not need to understand the full ECG waveform. We only need the **timing between heartbeats**. In normal rhythm those gaps are metronomic. In AF they scatter. So if we can find each heartbeat and measure the gaps, statistics describing how scattered those gaps are should reveal AF.

The gap between consecutive heartbeats is called an **RR interval** (named after the R wave, the tall spike of each beat). The study of how these intervals vary is **heart rate variability**, or HRV. That is the entire input to our models.

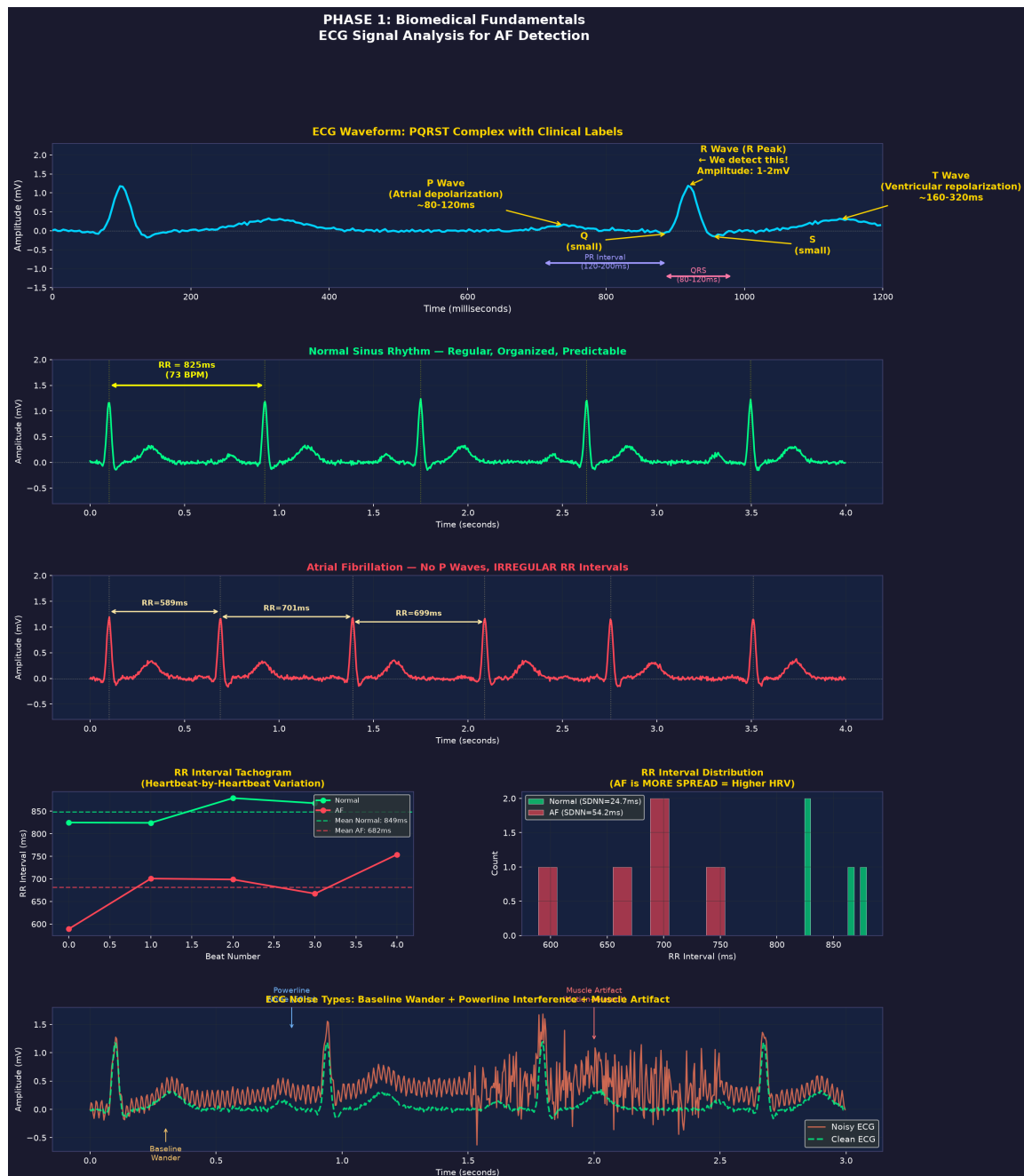


Figure 1.1 — The anatomy of an ECG waveform. The tall spike is the R peak; the gap between two R peaks is one RR interval.

Part 2 — The Dataset

CACHET-CADB is unusual and that is why it was chosen. Most ECG datasets give you the heart signal and nothing else. This one also records **what the person was physically doing** at every moment: movement intensity from an accelerometer, activity type, body position, step count, even sleep state.

That extra channel is what makes the research question possible. When our model gets an answer wrong, we can look up the circumstances and ask *why*.

How the files are organised

A recording is stored in a format called **Unisens**. Each folder holds several raw binary files plus one XML file describing them:

```
signal/P1/a2b3c4@cachet.dk1/0/  
  unisens.xml ← the metadata: how to decode everything else  
  ecg.bin     ← 60 MB of raw heart signal  
  acc.bin     ← accelerometer, 3 axes  
  press.bin   ← air pressure  
  
annotations/P1/a2b3c4@cachet.dk1/0/  
  annotation.csv ← Start, End, Class – the doctor's labels  
  context.xlsx  ← 32 columns of activity data
```

The subject folders are P1 to P19, P21, P23, plus PNSR-1, PNSR-3 and PNSR-4. **Note the gaps** — there is no P20 or P22. The PNSR subjects are healthy volunteers used as controls. P16 exists but has no usable labels, so we work with 23 subjects, not 24.

What a label looks like

Each row of annotation.csv marks one labelled stretch of ECG:

```
Start,End,Class  
32196064,32206304,1
```

Start and End are positions in the ecg.bin file, counted in samples. Every labelled stretch is exactly 10,240 samples long, which at 1024 samples per second is exactly **10 seconds**. So the dataset's natural unit of analysis is a 10-second window, and we respect that rather than inventing our own.

Class code	Meaning	Count	Share
1	AF (atrial fibrillation)	747	46.6%
2	NSR (normal sinus rhythm)	615	38.4%
3	Noise (signal unreadable)	221	13.8%
4	Other arrhythmia	19	1.2%

Notice class 3, **Noise**. These are segments the doctors could not read at all — the signal was too corrupted. There are 221 of them, and they will turn out to be the most important part of this entire project.

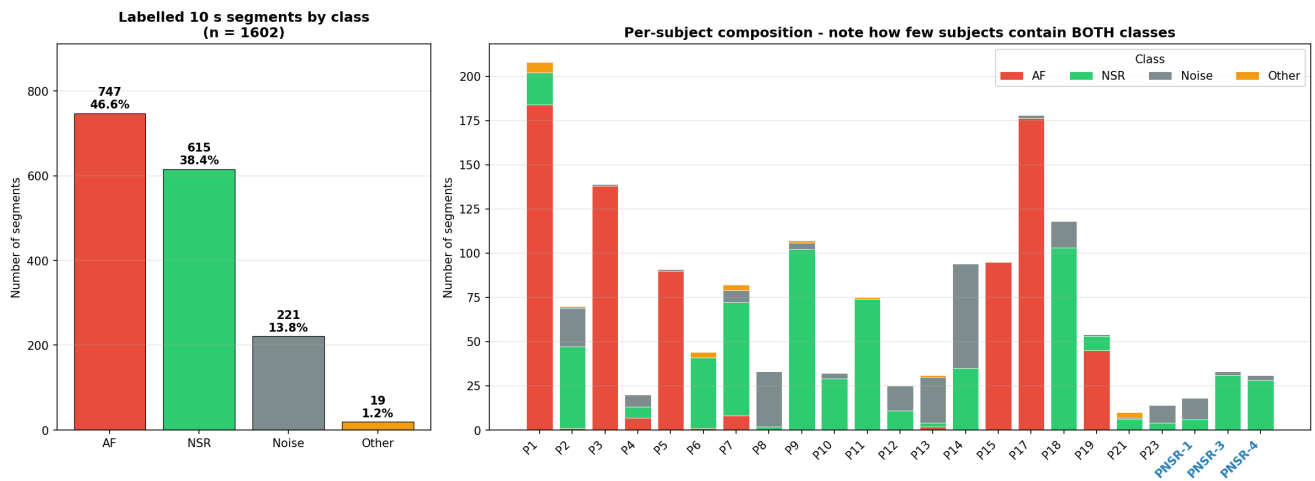


Figure 2.1 — Left: how many segments of each class. Right: composition per subject. Notice that most subjects are almost entirely one class — only 7 of 23 contain both AF and NSR. This causes real problems later.

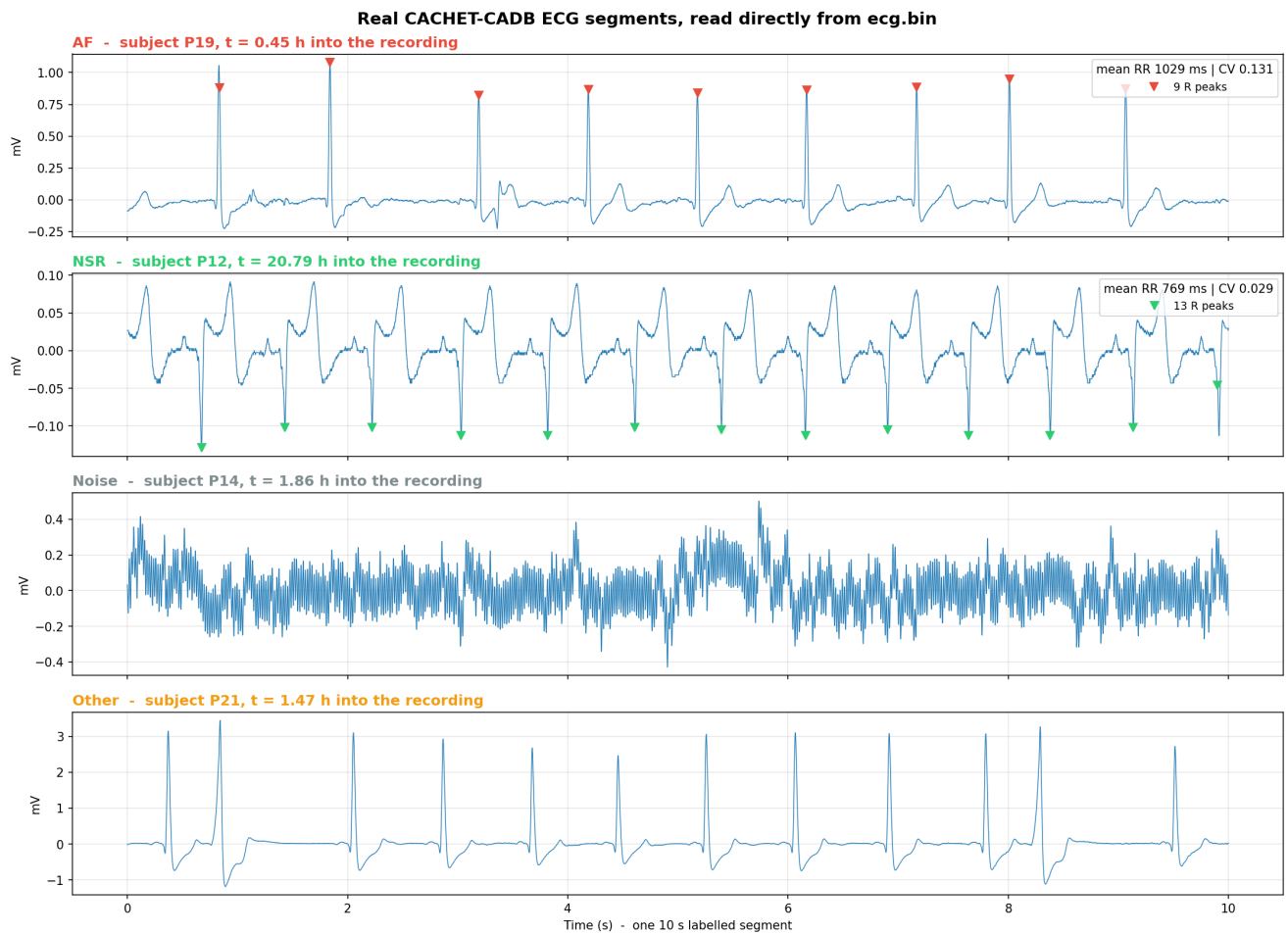


Figure 2.2 — Real segments straight from the dataset. In the AF panel the R peaks are unevenly spaced. In NSR they are metronomic. The Noise panel shows why 221 segments had to be excluded — there is no recoverable rhythm.

Part 3 — From Bytes to Numbers

This is the part that is invisible unless you go looking, and it is where the subtlest bugs hide.

Step 1: the file is just bytes

ecg.bin has no header. It is 60 megabytes of numbers packed end to end with nothing explaining them. On its own it is undecodable — you cannot even tell whether two bytes are one number or half of a bigger one.

Step 2: the XML is the decoder ring

unisens.xml supplies the missing instructions:

```
<signalEntry contentClass="ecg" sampleRate="1024"
  dataType="int16" baseline="2048"
  lsbValue="0.0026858" unit="mV" />
```

Read that as: each sample is a 16-bit whole number; there are 1024 of them per second; the value 2048 means zero volts; and each step of 1 equals 0.0026858 millivolts.

Steps 3 and 4: integers, then millivolts

We read the bytes as integers, then convert to real physical units:

```
millivolts = (raw_integer - 2048) × 0.0026858
```

How we checked this was right: we applied the same procedure to the accelerometer and got 0.993 g while the sensor sat still. Gravity is 1.0 g. Getting the right answer for a quantity we already know is the cheapest possible proof that the decoding is correct.

Steps 5 to 8: the reduction

From there, the data shrinks dramatically at every stage. Filtering removes frequencies that cannot be heart signal. Downsampling from 1024 to 256 samples per second discards detail we do not need. Peak detection collapses 2,560 numbers into about 12 heartbeat positions. Those become 11 intervals. Those 11 intervals become 20 summary statistics. Feature selection keeps 10.

The critical consequence: the model at the end only ever sees the final ten numbers. It never touches an ECG. If information is destroyed at step 6, no amount of clever modelling at step 8 can recover it — and that is exactly the trap we fell into, twice.

Part 4 — Cleaning the Signal

Raw ECG from a wearable contains three kinds of contamination, and each needs a different tool.

- **Baseline wander** — a slow drift as the person breathes and the electrodes shift. It lives below 0.5 Hz. We remove it with a **high-pass filter**, which blocks slow changes.
- **Powerline interference** — a 50 Hz hum picked up from mains electricity in the room. We remove it with a **notch filter**, which cuts one narrow frequency and leaves everything else.
- **High-frequency noise** — muscle activity and electrical crackle above 40 Hz. A **low-pass filter** blocks it.

Combining the high-pass and low-pass gives a **band-pass filter** that keeps only 0.5 to 40 Hz — the range where genuine ECG content lives.

Why we then downsample: 1024 samples per second is far more than ECG needs. A rule called the Nyquist theorem says you need at least twice the highest frequency of interest. Our highest is 40 Hz, so 256 samples per second is comfortably enough. Downsampling makes everything four times faster and loses nothing that matters.

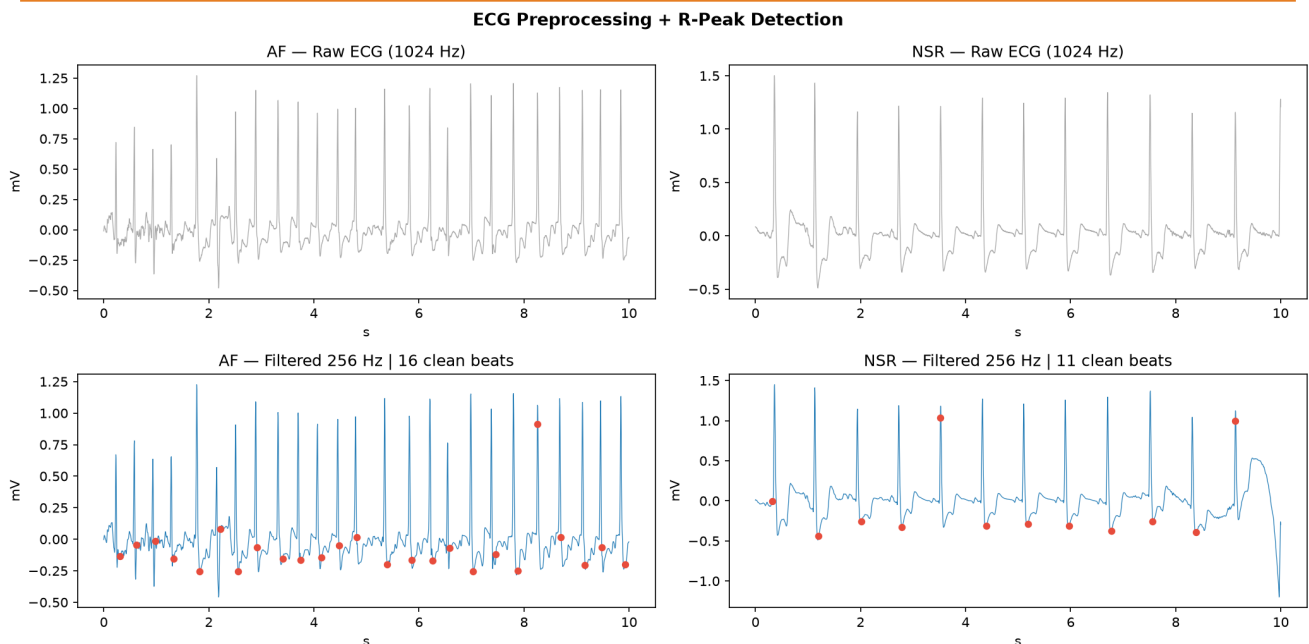


Figure 4.1 — The same ECG segment before and after cleaning. The drift flattens, the hum disappears, and the R peaks become far easier to locate reliably.

Part 5 — Finding the Heartbeats

With a clean signal we locate each beat. The classic method is called **Pan-Tompkins**, and it is understandable in four steps:

- **Band-pass 5 to 25 Hz.** The QRS complex (the tall spike) lives in this range. P waves, T waves and drift do not, so this alone removes most false candidates.
- **Square the signal.** This makes large deflections dominate and removes the sign, so upward and downward spikes both count.
- **Smooth over 150 milliseconds** — roughly one QRS width. Each sharp spike becomes a rounded bump that is easy to find.
- **Pick peaks, enforcing a 300 ms gap.** No human heart beats faster than 200 times per minute, so anything closer than 300 ms must be an artifact.

The output is a list of positions. Subtracting consecutive positions gives the **RR intervals** — the raw material for everything that follows.

Part 6 — Measuring Irregularity

From about 11 intervals we compute 24 statistics. They fall into four families.

Time-domain — direct measures of spread:

Feature	Plain-English meaning
MeanNN	Average gap between beats. $60000/\text{MeanNN}$ gives heart rate in bpm.
SDNN	Standard deviation of the gaps. Higher = more variable.
RMSSD	Typical size of the jump from one gap to the next. The key AF marker.
pNN50	Percentage of consecutive gaps differing by more than 50 ms.
CVNN	SDNN divided by MeanNN — variability relative to the heart rate.
MedNN, MadNN, IQRNN	Robust versions using medians instead of means.

Non-linear — geometric and information-theoretic views:

Feature	Plain-English meaning
SD1	Short-term scatter on a Poincare plot (each gap against the next).
SD2	Long-term scatter on the same plot.
SampEn / ApEn	How unpredictable the sequence is. AF is less predictable.
ShannonEn	How spread out the distribution of gaps is.

Fragmentation — PAS, PIP, IALS, PSS count direction changes in the interval sequence.

Rate morphology — ECG_Rate_Mean, Max, Min and Baseline. Every other feature measures the spread of intervals; these four measure the level and its extremes. They were added after reviewing Hasan & Motin (2025), and they matter here because a quarter of the AF in this dataset runs above 110 beats per minute, which dispersion alone does not capture.

What we deliberately excluded: frequency-domain features (LF, HF, LF/HF ratio) appear in most HRV papers, but they need at least 25 seconds of signal to resolve. Our windows are 10 seconds. Computing

them anyway would produce numbers that look scientific and mean nothing, so we leave them out and say so.

Part 7 — Five Bugs We Found

Each of these looked correct at first glance. Each would have quietly corrupted the results. This section is the most valuable part of the guide, because the debugging technique matters more than any single fix.

Bug 1: the sample rate was wrong

The configuration file said 256 samples per second. Every unisens.xml in the dataset says 1024. A wrong sample rate makes every heart rate and every interval wrong by a factor of four — and nothing crashes.

Lesson: trust the metadata file over any number typed by a human. Our loader now reads the rate from each recording and warns if it disagrees with the configuration.

Bug 2: the class labels were backwards

We assumed 1 meant normal and 2 meant AF, because normal usually comes first. That assumption was wrong. This is the most dangerous class of bug there is: the pipeline runs perfectly, the metrics look plausible, and every single prediction is inverted.

We proved the correct mapping three independent ways rather than trusting any one of them:

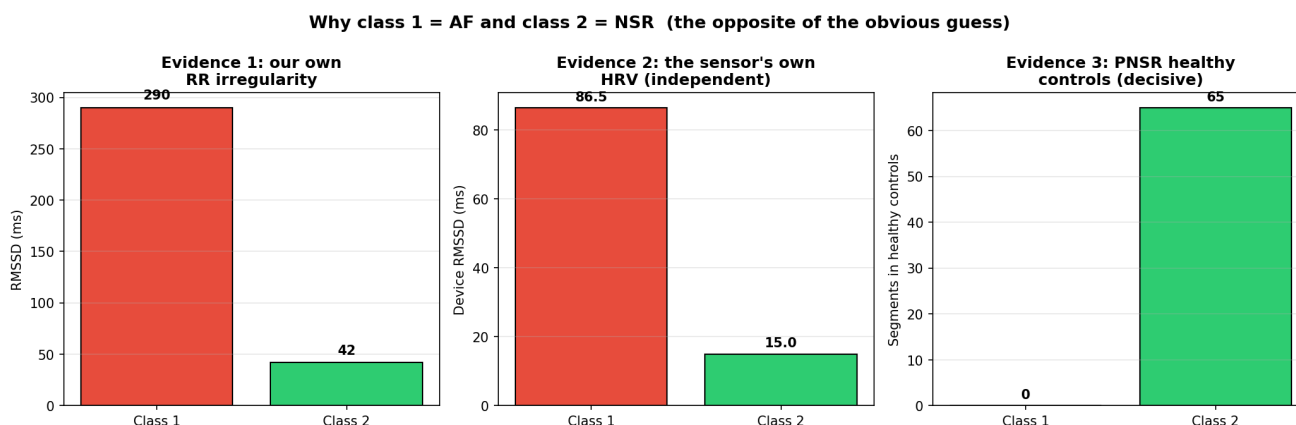


Figure 7.1 — Three independent lines of evidence. Left: our own measurement of interval irregularity. Middle: the sensor firmware's own HRV calculation, computed without our code. Right: the healthy control subjects, who have zero class-1 segments. A healthy volunteer cannot be in atrial fibrillation, so class 2 must be NSR.

Lesson: when a labelling convention is not documented, verify it against something you already know to be true. The healthy control cohort was the decisive test and it cost ten minutes.

Bug 3: the window length was wrong

The configuration used 5-minute windows because that is the clinical standard for HRV. But the dataset's own labels are 10 seconds long. Merging them into 5-minute windows would have combined segments with conflicting labels, destroying the very thing we are trying to learn.

Bug 4: the ectopic filter was deleting the answer

This was the serious one. Standard HRV practice removes **ectopic beats** — intervals that deviate more than about 20% from their neighbours — on the assumption that such a jump is an error rather than real physiology.

That assumption is correct for normal rhythm and **catastrophic for AF**. In atrial fibrillation consecutive intervals genuinely do differ by more than 20%. That irregularity is not noise to be cleaned — it is the diagnosis. The filter was deleting the evidence and then asking the classifier to find it.

Bug 3: the ectopic beat filter was deleting the AF signature

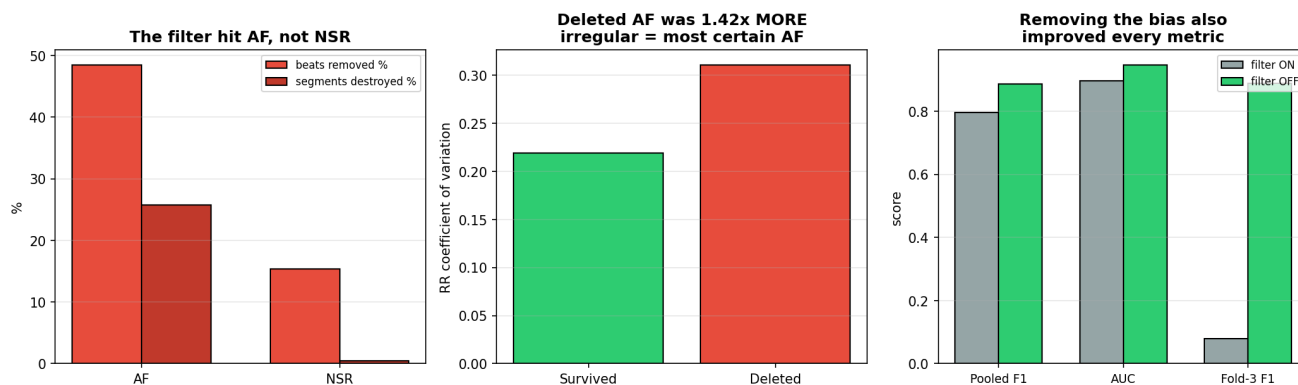


Figure 7.2 — Left: the filter removed 48.5% of AF beats but only 15.4% of NSR beats, and destroyed a quarter of all AF segments. Middle: the deleted segments were 1.42 times more irregular than the survivors, meaning it removed the most unambiguous AF. Right: switching it off improved every metric.

The middle panel is the one that matters scientifically. Deleting the *most certainly AF* segments is **selection on the outcome** — it makes the remaining problem artificially easy and inflates every reported number. A detector validated that way looks excellent on paper and fails on precisely the cases that matter clinically.

Result of the fix: pooled F1 rose from 0.795 to 0.887, AUC from 0.897 to 0.947, and the variation between cross-validation folds collapsed from ± 0.332 to ± 0.071 . It is rare for removing a bias to also improve performance; here it did both.

Bug 5: the feature selection did nothing

The pipeline selected the top 20 features from a set of exactly 20. It was reporting a selection step that selected everything. Reducing it to 10 made the stage meaningful — and revealed something useful, covered in Part 9.

Part 8 — Training the Models

We train three classifiers. An **SVM** (support vector machine) draws the best boundary between the two classes in the feature space. **XGBoost** builds hundreds of small decision trees, each correcting the errors of the last. **Logistic Regression** fits a simple weighted sum of the features and is included as an honest baseline: if a linear model nearly matches the other two, the extra machinery is not earning its place, and saying so is more useful than reporting only the winner.

The rule that keeps us honest

If you split the data randomly, segments from the same patient land in both the training and test sets. The model then recognises *the patient* rather than *the disease*, and reports excellent accuracy that would collapse on a new person. This is called **data leakage**.

We use **GroupKFold**, which guarantees that every segment from one subject stays together. If a subject is in the test set, none of their data was used for training. This produces lower numbers than a random split, and they are the honest ones.

Handling imbalance

We apply **SMOTE**, which generates synthetic examples of the minority class to even out the training data. It is applied *only inside each training fold*, never to the test fold — otherwise synthetic data would leak into evaluation.

Feature selection with MRMR

MRMR stands for Minimum Redundancy Maximum Relevance. It picks features that are individually informative but not repetitive. From 24 features it kept 10: pNN50, MCVNN, CVNN, ShannonEn, pNN20, IQRNN, MadNN, SD1, RMSSD and SDNN.

Every survivor measures the spread of RR intervals. All the fragmentation features were dropped, and all entropies except Shannon. AUC did not change at all. That is a genuine finding: the standard HRV feature set is highly redundant, and a wearable could compute half as much for the same accuracy.

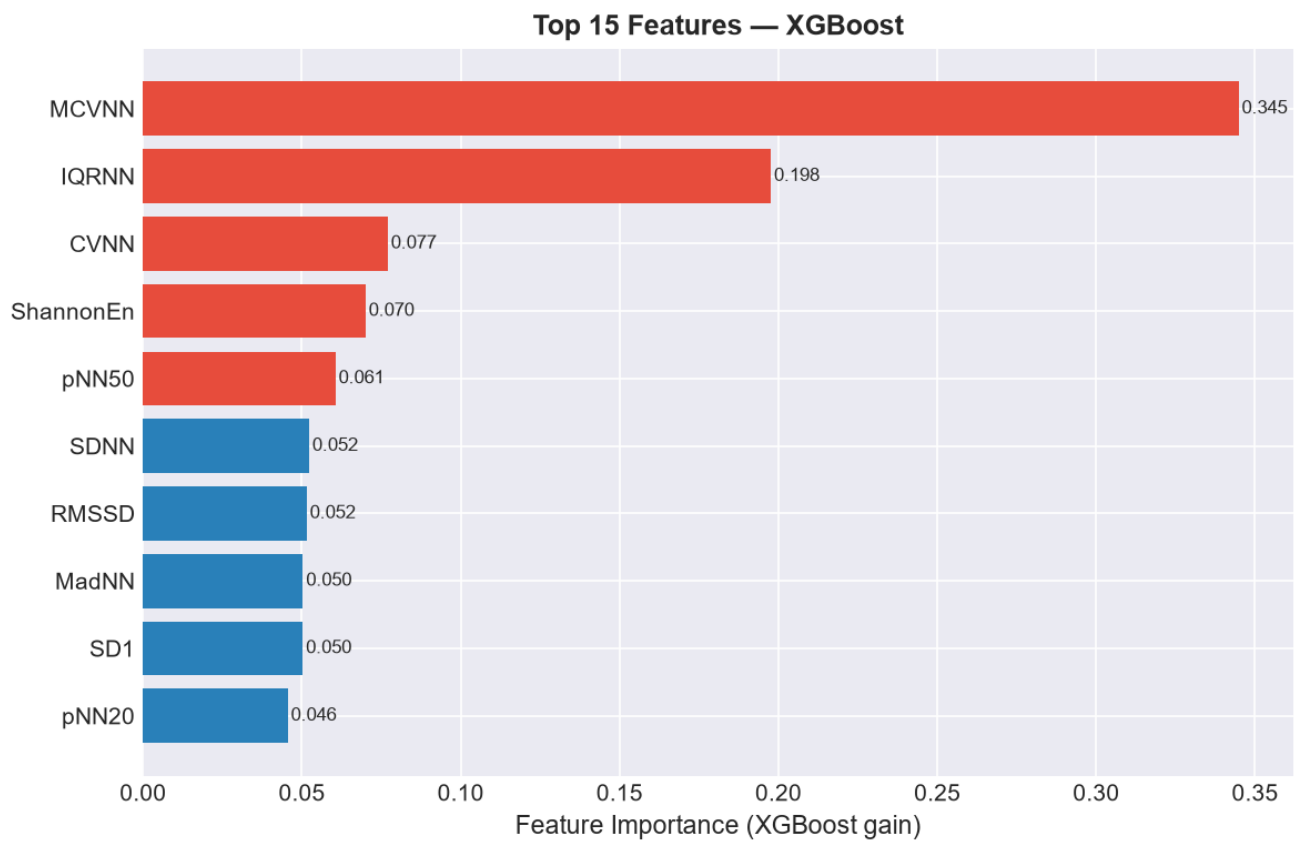


Figure 8.1 — Which features XGBoost actually relied on. MCVNN and IQRNN dominate; both are robust measures of interval spread.

Part 9 — The Results

Metric	SVM	XGBoost	What it means
Accuracy	0.877	0.887	Share of all segments called correctly
F1	0.880	0.891	Balance of precision and recall
AUC	0.932	0.923	Ability to rank AF above non-AF
Sensitivity	0.847	0.884	Share of real AF that was caught
Specificity	0.908	0.883	Share of normal correctly left alone

How to read AUC: 0.5 means guessing; 1.0 is perfect. Around 0.93 means that if you pick one AF segment and one normal segment at random, the model gives the AF one a higher score about 93 times out of 100.

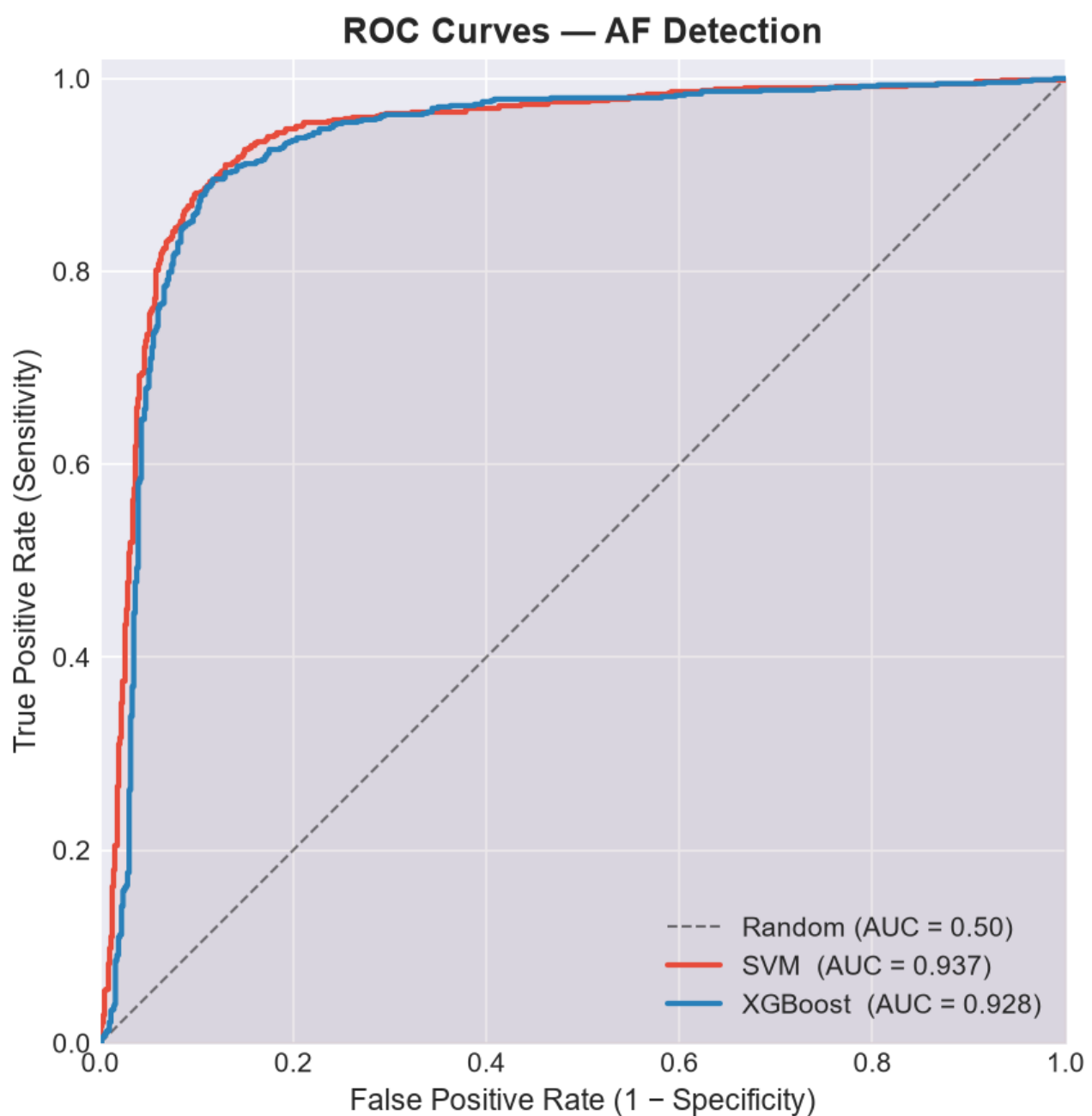


Figure 9.1 — ROC curves. Each point is a different decision threshold, trading catching more AF against raising more false alarms. The further the curve bows toward the top-left, the better.

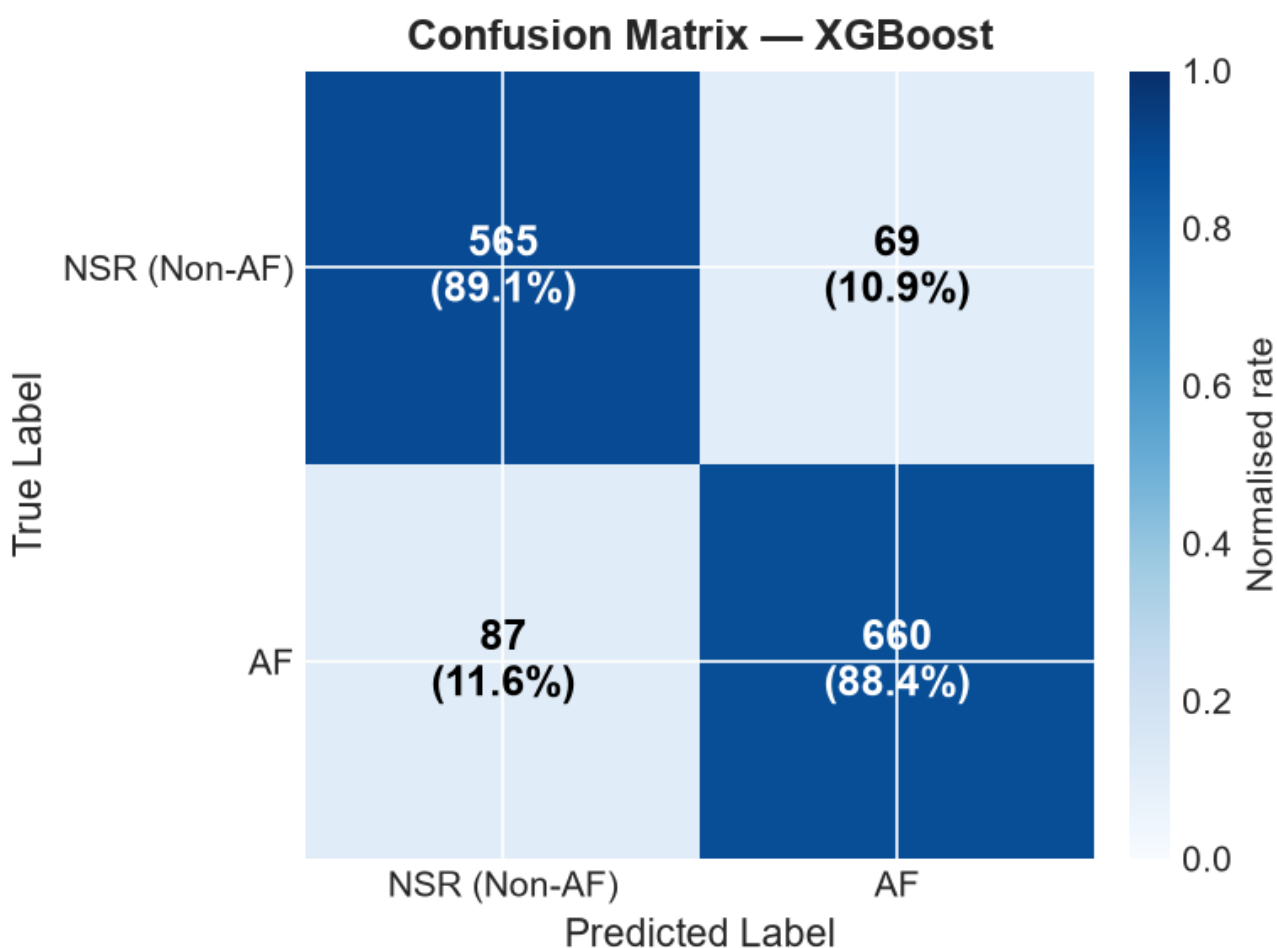


Figure 9.2 — The confusion matrix. Rows are truth, columns are prediction. The diagonal is correct; off-diagonal cells are the two kinds of mistake: missed AF (dangerous) and false alarms (costly and alarming).

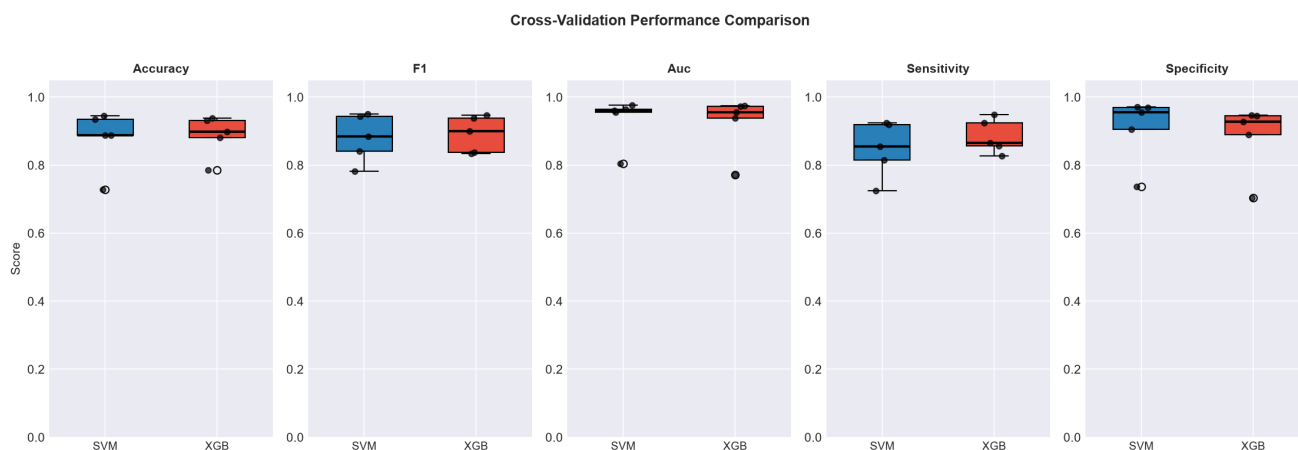


Figure 9.3 — Performance across the five cross-validation folds. The spread matters as much as the average: fold 1 is much weaker than the rest, because the subjects who land in it are unusually hard.

How often is an AF alarm correct?

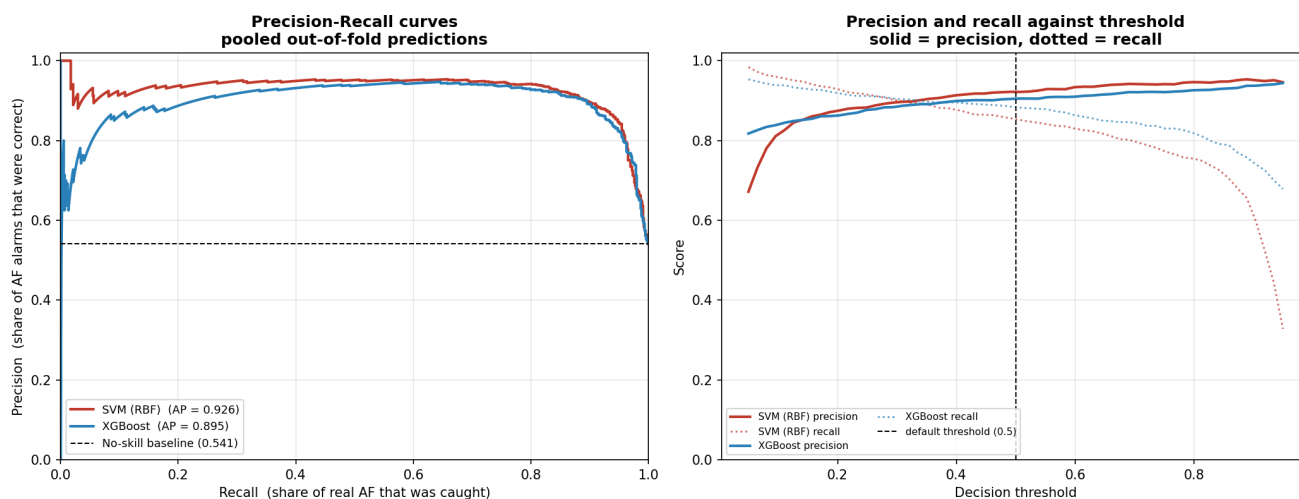


Figure 9.4 — Precision-Recall curves. These answer the question a patient cares about: when the device says AF, how often is it right? The dashed line is the no-skill baseline at the AF prevalence (~0.54), not 0.5 — compare against that.

An important caution on reporting. Averaging the five folds gives a misleading picture, because the folds contain very different mixes of patients. The honest figure is the **pooled out-of-fold** result, computed across every prediction at once: F1 0.887, AUC 0.947 for the SVM.

Part 10 — Why It Fails (The Real Contribution)

A model with AUC 0.93 sounds finished. It is not. The interesting question is not how often it is right, but **what makes it wrong** — and this dataset is one of the very few that can answer that.

The first attempt, which found nothing

The original plan: among normal segments the model wrongly flagged as AF, was it misled by physical activity? The statistical tests came back empty — chi-square $p = 0.65$, Mann-Whitney $p = 0.14$.

That null is not a finding, and here is why. Among the 634 normal segments, movement acceleration had a median of 0.008 g, and only **31 segments out of 634** showed meaningful movement at all. You cannot test whether movement explains errors when almost nothing moves.

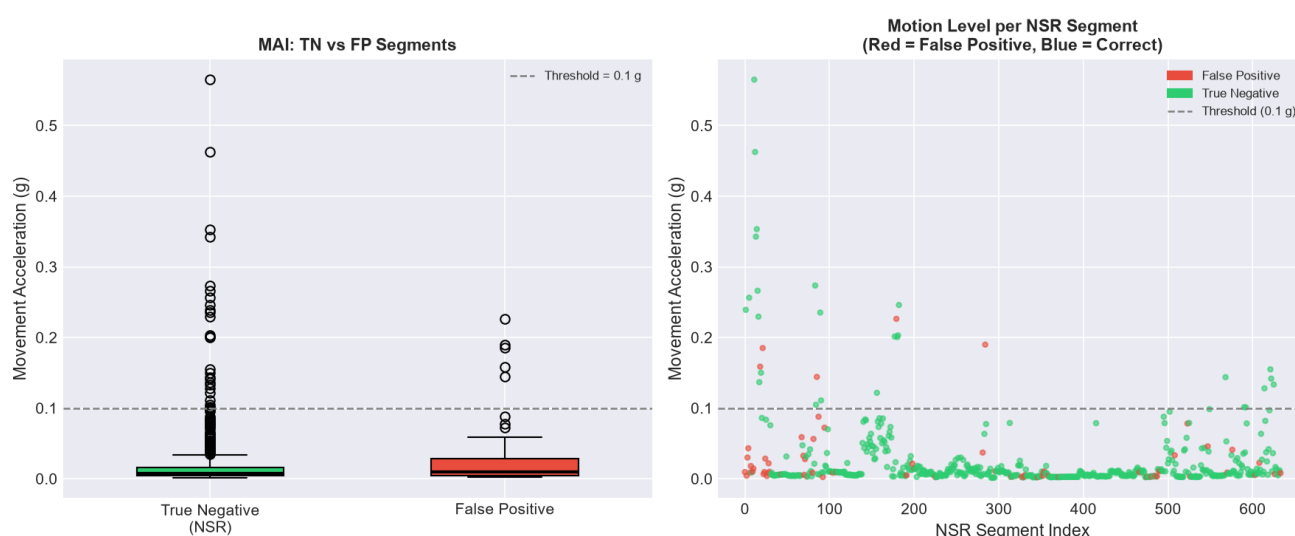


Figure 10.1 — The original analysis. The movement values are clustered near zero, leaving almost no variation for a statistical test to work with.

Why the movement was missing — and where it went. Doctors annotate segments they can *read*. When motion corrupts a recording the rhythm becomes uninterpretable, so it is not labelled NSR — it is labelled **Noise**. All 221 of them. The motion-corrupted population had been sorted into a separate class and then excluded from the analysis. We were looking in the one place the effect could not appear.

The second attempt, which found a great deal

So we scored the trained model on all 221 Noise segments. Every AF prediction there is a false alarm by definition: an unreadable segment contains no rhythm to detect, so announcing atrial fibrillation on one is simply wrong.

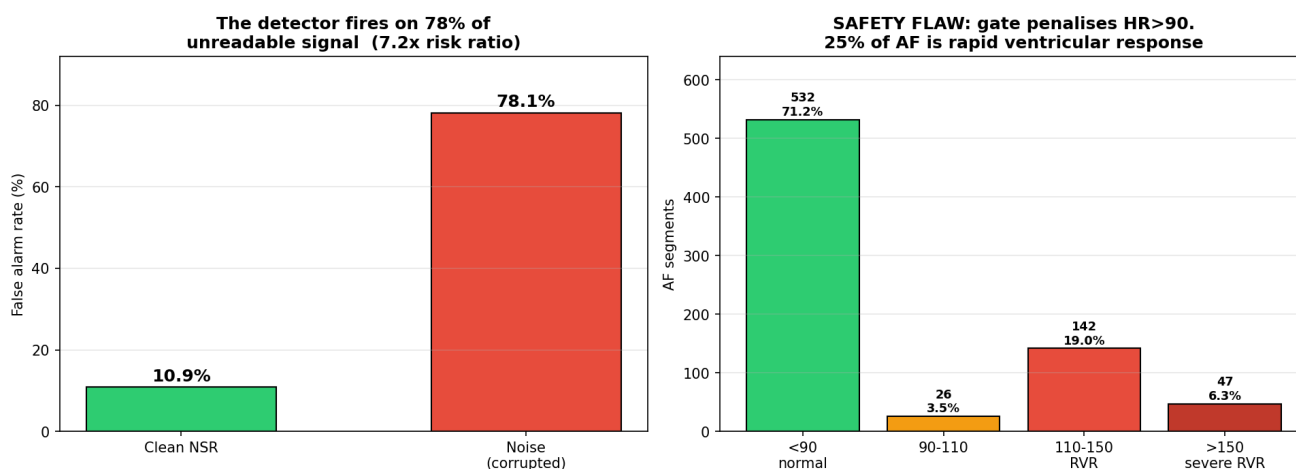


Figure 10.2 — Left: the headline result. The detector fires on 78% of unreadable segments versus 11% of clean ones — a sevenfold increase. Right: a related danger discussed in Part 11.

The detector raises a false alarm on roughly four out of every five corrupted segments. For a wearable worn during ordinary life, that is the difference between a useful device and one the patient switches off.

Why this happens — the mechanism

We verified the cause rather than speculating. Corrupted signal makes the peak detector find beats that are not there and miss beats that are. That produces erratic RR intervals. And erratic RR intervals are exactly the signature of AF.

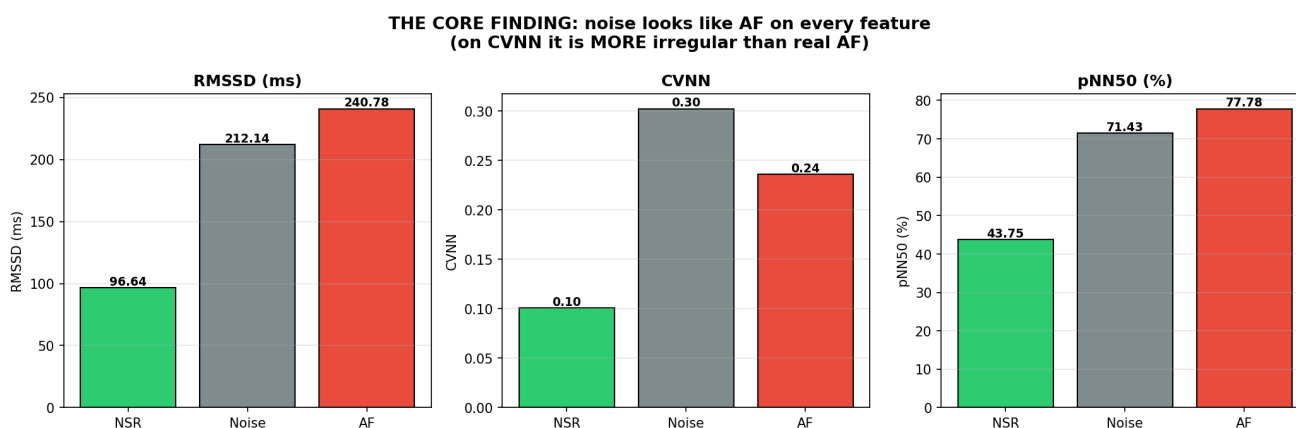


Figure 10.3 — Noise sits alongside AF on every feature. On CVNN it is **more irregular than genuine atrial fibrillation**. The classifier has no way to tell them apart, because on the numbers it sees, they are the same thing.

This reframes the entire project. The limitation is not the classifier, and not the choice of features. Artifact-induced irregularity and fibrillation-induced irregularity are **mathematically identical** once you reduce the signal to interval timings.

What must not be claimed. Within the Noise segments, movement did not predict which ones triggered alarms (Spearman rho = -0.066, p = 0.33). Do not write that movement is irrelevant. When 80% of segments alarm, the outcome is nearly constant, so no variable can correlate with it. The test is **saturated**, not informative — a different statement entirely.

Part 11 — Attempting a Fix

If corrupted signal causes false alarms, the obvious response is a **quality gate**: check whether the signal is good enough to judge before letting the classifier speak. If not, refuse to answer.

A gate was built using measures already available — implied heart rate, interval variability, and beat count. It swept a threshold from lenient to strict, measuring what is gained and lost.

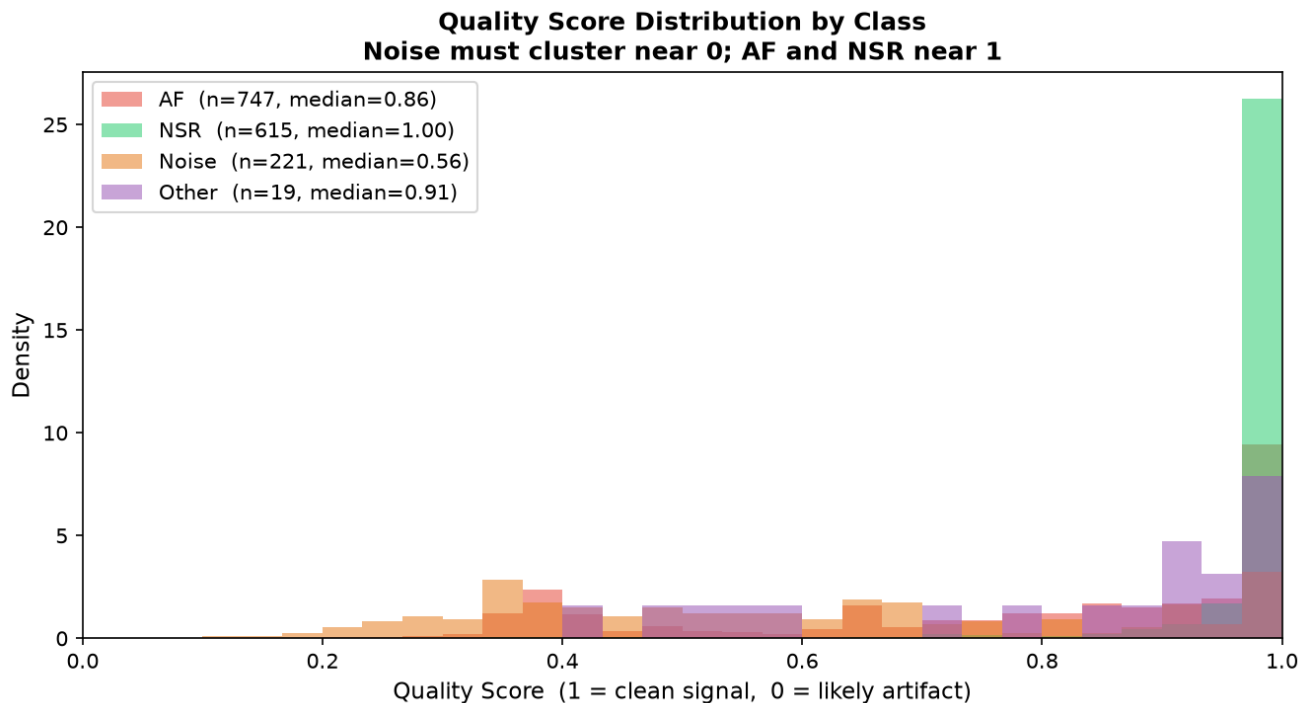


Figure 11.1 — Quality scores by class. Noise does score lower than clean signal, and the separation is statistically significant.

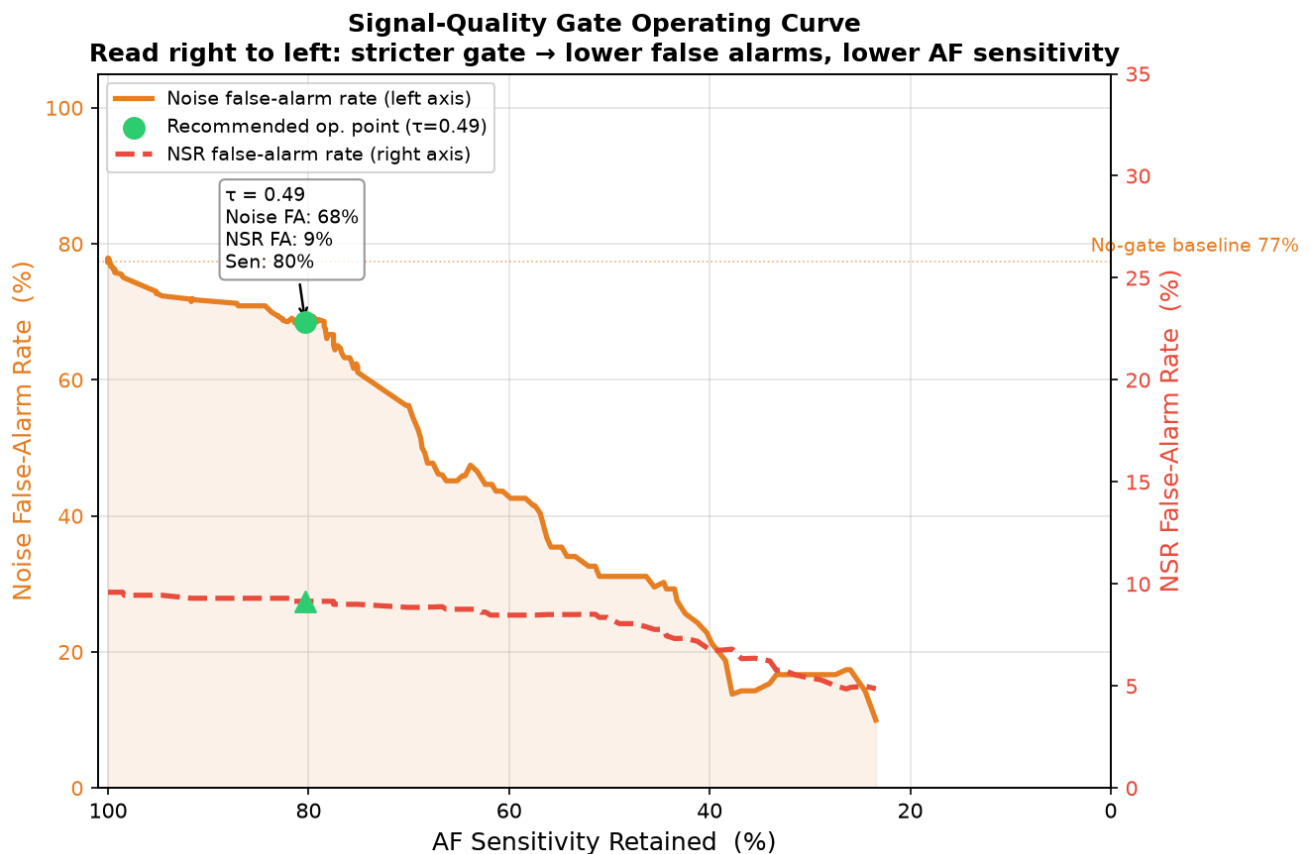


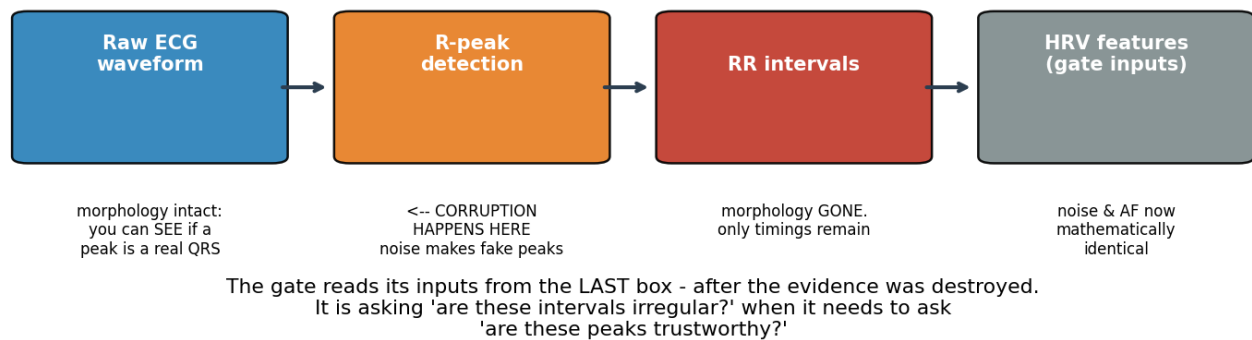
Figure 11.2 — The operating curve. Every point is one threshold setting. Rejecting enough noise to matter means discarding a large share of genuine AF.

The numbers are discouraging. At a moderate setting, rejecting 42% of noise cuts false alarms only from 77% to 69%, while already losing 20% of real AF. To push false alarms below 50% you must discard roughly a third of all genuine AF.

Why the gate cannot work — and this is provable

The gate's inputs are all derived from RR intervals. RR intervals are computed *after* peak detection. But the corruption happens *during* peak detection. By the time you hold interval statistics, the artifact has already been converted into fake irregularity indistinguishable from AF.

Why the RR-based quality gate cannot work



FIX: bSQI - run two different peak detectors on the RAW waveform and measure agreement. Clean: they agree. Noise: they disagree. Real AF: they STILL agree (peaks are real, just irregular).

Figure 11.3 — The gate reads its evidence from a stage after that evidence was destroyed. It asks whether the intervals are irregular, when it needs to ask whether the peaks are trustworthy.

So the poor trade-off is not an empirical limit that further tuning might improve. It is **built in by construction** the moment those inputs were chosen. Two things cannot be separated using only the axis on which they coincide.

A safety flaw worth catching

The gate's strongest penalty is on high heart rate. But a quarter of the AF segments in this dataset run above 110 beats per minute — that is AF with **rapid ventricular response**, clinically the most urgent presentation and the one most likely to need immediate treatment.

A gate that penalises fast rhythms therefore discards preferentially the AF cases you least want to miss. It is safest on paper and most dangerous in deployment. This must be fixed or explicitly disclosed.

The fix that would work: bSQI

Run **two different peak detectors** over the same raw waveform and measure how much they agree.

- **Clean signal:** both find the same peaks — high agreement.
- **Noise:** they disagree, because each is fooled differently.
- **Genuine AF:** they still agree. The peaks are real QRS complexes, merely irregularly spaced.

That last line is the whole point. bSQI asks whether the peaks are *trustworthy*, not whether the intervals are *irregular* — a question with different answers for noise and AF. It works on the raw waveform, before the information is lost.

Part 12 — Where The Project Stands

Completed

Stage	Status	Outcome
Dataset indexing	Done	1,602 segments located and validated
Unisens decoding	Done	Verified against gravity (0.993 g)
Preprocessing	Done	Band-pass, notch, downsample to 256 Hz
R-peak detection	Done	Pan-Tompkins via NeuroKit2
Feature extraction	Done	20 HRV features, 1,600 of 1,602 succeeded
Feature selection	Done	10 features, no loss of AUC
Model training	Done	SVM AUC 0.932, XGBoost 0.923
FP analysis (clean)	Done	Null result — serves as control
FP analysis (noise)	Done	78-84% false alarms — main finding
Quality gate	Done	Shown to be structurally incapable

Outstanding, in priority order

- **Implement bSQI** on the raw waveform. This would turn Part 11 from a negative result into a demonstrated solution, and is the single highest-value remaining task.
- **Diagnose fold 1**, where AUC drops to 0.77 against 0.95+ elsewhere. Identify which subjects land there and why they are hard.
- **External validation** on MIT-BIH AFDB, already downloaded. Tests whether the model generalises beyond this one dataset.
- **Write-up**. There is already enough here for a strong paper.

What the methodology section must disclose

- P16 has no usable labels — 23 subjects, not 24.
- 221 Noise segments excluded from training but retained for the false-positive analysis.
- 19 'Other' arrhythmia segments grouped with non-AF; too few to model separately.
- Only 7 of 23 subjects contain both AF and NSR, which is why fold composition varies so much.
- Frequency-domain HRV excluded — a 10-second window cannot resolve it.
- The ectopic filter is deliberately disabled, with the evidence reproducible via the test script.

The one-paragraph summary

An HRV-based atrial fibrillation detector reaching AUC 0.93 on curated data produces false alarms on roughly 80% of motion-corrupted segments, because artifact-induced RR irregularity is mathematically indistinguishable from fibrillation. A quality gate built from RR-derived statistics cannot resolve this, since its inputs are computed after the distinguishing information has been destroyed. Signal-quality assessment at the waveform level — not richer features or better classifiers — is the binding constraint on real-world deployment.

Part 12b — How This Compares to Published Work

A useful reference point is Hasan & Motin, *Automated Classification of Atrial Fibrillation from ECG Signals Using Machine Learning Techniques*, IEEE SPICSCON 2025. They tackle the same task with a very similar method, which makes the differences instructive.

	Hasan & Motin (2025)	This project
Dataset	CPSC 2021	CACHET-CADB
Recordings used	30 (of 1,436 available)	1,602 segments, 23 subjects
Recording setting	Clinical	Free-living ambulatory
Window length	20 seconds	10 seconds
Classifiers	SVM, LR, XGBoost	SVM, LR, XGBoost
Splitting	Not stated	GroupKFold by subject
Reported accuracy	95.4%	88.7%
Reported AUC	~0.99	0.932
Noise segments	Not evaluated	78-84% false alarm rate

An independent confirmation worth citing

The two projects agree on which features matter, despite using different datasets, different window lengths and different populations:

Rank	Hasan & Motin	This project
1	MCVNN (0.37)	MCVNN (0.345)
2	MadNN (0.23)	IQRNN (0.198)
3	IQRNN (0.06)	CVNN (0.077)

MCVNN ranks first in both, at almost the same importance value. MCVNN and IQRNN appear in both top-threes. Two independent teams reached the same conclusion: **robust, median-based measures of RR-interval spread carry the diagnostic signal**. This also vindicates our MRMR result, which kept exactly that family and discarded the fragmentation features.

Where we deliberately diverge

Their feature set includes frequency-domain measures — ULF, VLF, LF, HF, VHF, total power and the LF/HF ratio — extracted from 20-second windows, plus SDANN1/2/5 and SDNNI1/2/5, which are *defined* as standard deviations over 1, 2 and 5-minute windows.

You cannot resolve a frequency whose period is longer than your window. ULF sits below 0.003 Hz, so one cycle takes over five minutes; LF needs at least 25 seconds. In a 20-second window these quantities are not measurable. NeuroKit2 returns a number regardless, but that number describes the algorithm's behaviour rather than the patient's physiology.

We therefore exclude frequency-domain features and state why. We did adopt their four rate-morphology features, which are simple arithmetic on RR intervals and perfectly valid at 10 seconds, along with their Logistic Regression baseline and their use of Precision-Recall curves.

Why the accuracy gap is not a weakness

Their 95.4% and AUC 0.99 come from 30 hand-picked recordings out of 1,436 available, in a clinical setting, with no stated subject-wise splitting. Our 88.7% and AUC 0.932 come from free-living ambulatory data with strict subject-wise validation, where a subject in the test set contributed nothing to training. These are not the same problem, and ours is the harder one.

The gap that defines our contribution. That paper reports high accuracy on curated data and stops. It never asks what happens on corrupted signal. Our finding — that a detector at this performance level raises false alarms on roughly 80% of motion-corrupted segments — is exactly the question such work leaves open. Suggested framing for the introduction:

“Prior work reports 95.4% accuracy and AUC 0.99 for HRV-based AF classification on curated recordings [Hasan & Motin, 2025]. Such evaluations exclude the motion-corrupted segments that dominate free-living wearable data. We show that a detector of comparable performance produces false alarms on 78-84% of such segments, and that the cause is a structural confound between artifact-induced and fibrillation-induced RR irregularity.”

Part 13 — Glossary

Term	Plain-English meaning
Atrial fibrillation (AF)	Chaotic quivering of the heart's upper chambers, producing irregular beats.
NSR	Normal sinus rhythm — the healthy, regular pattern.
ECG	Electrocardiogram; a recording of the heart's electrical activity.
R peak	The tall spike marking one heartbeat.
RR interval	The time between two consecutive R peaks.
HRV	Heart rate variability — how much RR intervals vary.
RMSSD	Typical jump between consecutive RR intervals. Key AF marker.
SDNN	Standard deviation of RR intervals.
CVNN	SDNN divided by mean RR; variability relative to heart rate.
pNN50	Percentage of consecutive intervals differing by over 50 ms.
Poincare plot	Each RR interval plotted against the next. AF forms a diffuse cloud.
SD1 / SD2	Short-term and long-term scatter on the Poincare plot.
Sample entropy	How unpredictable a sequence is.
Sampling rate	Measurements taken per second. Here 1024 Hz, reduced to 256 Hz.
Nyquist theorem	You must sample at over twice the highest frequency of interest.
Band-pass filter	Keeps a range of frequencies, blocks the rest.
Notch filter	Removes one narrow frequency, such as 50 Hz mains hum.
Ectopic beat	An irregular beat, normally treated as an artifact — but see Part 7.
Pan-Tompkins	The classic algorithm for locating R peaks.
SVM	A classifier that draws the best boundary between two classes.
XGBoost	A classifier combining many small decision trees.
MRMR	Feature selection favouring informative, non-redundant features.
Cross-validation	Repeated train/test splits to test reliability.
GroupKFold	Cross-validation keeping all of one subject's data together.
Data leakage	Test information reaching the model during training. Inflates results.
SMOTE	Generates synthetic minority-class examples to balance training data.
Sensitivity	Share of real AF correctly caught.
Specificity	Share of normal rhythm correctly left alone.
Precision	Of segments flagged as AF, the share that really were.
F1 score	The balance between precision and sensitivity.
AUC	Ranking ability. 0.5 = guessing, 1.0 = perfect.
ROC curve	Trade-off between catching AF and raising false alarms.
Confusion matrix	Table of correct and incorrect predictions by type.
False positive	Normal rhythm wrongly flagged as AF.
False negative	Real AF that was missed. The dangerous error.
Chi-square test	Tests whether two categorical variables are related.

Mann-Whitney U	Tests whether two groups differ, without assuming a bell curve.
Spearman rho	Measures whether two variables rise and fall together.
p-value	Probability of seeing this result by chance. Below 0.05 is conventional.
Effect size	How large a difference is, as opposed to how certain.
MAI	Movement acceleration index — how much the person was moving.
RVR	Rapid ventricular response — AF with a dangerously fast rate.
SQL	Signal quality index — a measure of how trustworthy a recording is.
bSQL	An SQL comparing two peak detectors' agreement.
Average Precision	Area under the Precision-Recall curve. Floor is the prevalence, not 0.5.
Logistic Regression	A linear classifier producing one signed weight per feature.
ECG_Rate_Max/Min	Fastest and slowest instantaneous heart rate inside one segment.
Unisens	The binary data format used by this dataset.